



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

ASSIGNMENT 02: Greedy & Dynamic Programming

Q1: Check if it is possible to survive on an Island. (Greedy)

Jarif got stuck on an island. There is only one shop on this island, and it is open on all days of the week except for Sunday. Consider following constraints:

N – The maximum unit of food you can buy each day.

S – Number of days you are required to survive.

M – Unit of food required each day to survive.

Currently, it's Monday, and he needs to survive for the next 'S' days.

Find the minimum number of days on which you need to buy food from the shop so that he can survive the next 'S' days or determine that it isn't possible to survive.

Example 1:

Input: S = 10, N = 16, M = 2

Output: 2

Explanation: One possible solution is to buy a box on the first day (Monday), it's sufficient to eat from this box up to 8th day (Monday) inclusive. Now, on the 9th day (Tuesday), you buy another box and use the chocolates in it to survive the 9th and 10th day.

Example 2:

Input: S = 10, N = 20, M = 30

Output: -1

Explanation: She can't survive even if she buy food because the maximum number of units she can buy in 1 day is less the required food for 1 day.

Primary Task: Create a function minimumDays() which takes S, N, and M as input parameters and returns the minimum number of days Ishika needs to buy food. Otherwise, returns -1 if she cannot survive.

Constraints:

$1 \leq N, S \leq 50$ $1 \leq M \leq 30$



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

Q2: Chocolate Distribution Problem (Greedy)

Given an array $A[]$ of positive integers of size N , where each value represents the number of chocolates in a packet. Each packet can have a variable number of chocolates. There are M students, the task is to distribute chocolate packets among M students such that :

1. Each student gets exactly one packet.
2. The difference between maximum number of chocolates given to a student and minimum number of chocolates given to a student is minimum.

Example 1:

Input: $N = 8, M = 5$

$A = \{3, 4, 1, 9, 56, 7, 9, 12\}$

Output: 6

Explanation: The minimum difference between maximum chocolates and minimum chocolates is $9 - 3 = 6$ by choosing following M packets: $\{3, 4, 9, 7, 9\}$.

Example 2:

Input: $N = 7, M = 3$

$A = \{7, 3, 2, 4, 9, 12, 56\}$

Output: 2

Explanation: The minimum difference between maximum chocolates and minimum chocolates is $4 - 2 = 2$ by choosing following M packets: $\{3, 2, 4\}$.

Primary Task: Create a function `findMinDiff()` which takes array $A[]$, N and M as input parameters and returns the minimum possible difference between maximum number of chocolates given to a student and minimum number of chocolates given to a student.

Constraints:

$$1 \leq T \leq 100 \quad 1 \leq N \leq 105$$

$$1 \leq A_i \leq 109 \quad 1 \leq M \leq N$$



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

Q3: Minimum Cost Climbing Stairs (DP)

You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

Input: cost = [10,15,20]

Output: 15

Explanation: You will start at index 1.

- Pay 15 and climb two steps to reach the top.

The total cost is 15.

Example 2:

Input: cost = [1,100,1,1,1,100,1,1,100,1]

Output: 6

Explanation: You will start at index 0.

- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.
- Pay 1 and climb one step to reach the top.

The total cost is 6.

Constraints:

$2 \leq \text{cost.length} \leq 1000$

$0 \leq \text{cost}[i] \leq 999$



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

Q4: Cut Into Segments (DP)

You are given an integer 'N' denoting the length of the rod. You need to determine the maximum number of segments you can make of this rod provided that each segment should be of the length 'X', 'Y' or 'Z'.

Input Format:

The first line of input contains an integer 'T' denoting the number of test cases.

The next 'T' lines represent the 'T' test cases.

Each test case contains 4 integers denoting 'N', 'X', 'Y', and 'Z', where 'N' is the length of the rod and 'X', 'Y', 'Z' are the segments into which a given rod can be cut into.

Output Format:

For each test case, return the maximum number of cut segments from the given rod.

Print the output of each test case in a separate line.

Constraint:

$$1 \leq T \leq 50$$

$$1 \leq N \leq 10000$$

$$1 \leq X, Y, Z \leq N$$

Sample Input 1:

```
2
7 5 2 2
8 3 3 3
```

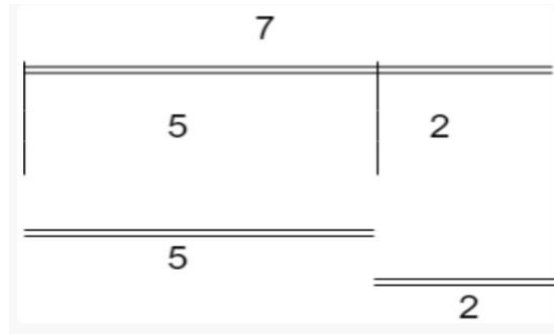
Sample Output 1:

```
2
0
```

Explanation For Sample Input 1:



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S



In the first test case, cut it into 2 parts of 5 and 2.

In the second case, there is no way to cut into segments of 3 length only as the length of the rod is less than the given length.

Sample Input 2:

2
7 3 2 2
8 1 4 4

Sample Output 2:

3
8

Explanation:

In the first test case, cut it into 3 parts of 3, 2 and 2.

In the second case, cut it into 8 parts of length 1.



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

Q5: Maximum Sum of Non-Adjacent Elements (DP)

You are given an array/list of 'N' integers. You are supposed to return the maximum sum of the subsequence with the constraint that no two elements are adjacent in the given array/list.

Note: A subsequence of an array/list is obtained by deleting some number of elements (can be zero) from the array/list, leaving the remaining elements in their original order.

Input format:

The first line contains a single integer 'T' denoting the number of test cases.

The first line of each test case contains a single integer 'N' denoting the number of elements in the array.

The second line contains 'N' single space-separated integers denoting the elements of the array/list.

Output format:

For each test case, print a single integer that denotes the maximum sum of the non-adjacent elements.

Print the output of each test case in a separate line.

Constraints:

$$1 \leq T \leq 500$$

$$1 \leq N \leq 1000$$

$$0 \leq \text{ARR}[i] \leq 10^5$$

Where 'ARR[i]' denotes the i^{th} element in the array/list

Sample Input 1:

2

3

1 2 4

4

2 1 4 9

Sample Output 1:

5

11



UNITED INTERNATIONAL UNIVERSITY
Department of Computer Science and Engineering (CSE)
Data Structure and Algorithms II Lab (CSE 2218)
Summer – 2026 | Section F, P, Q, S

Explanation to Sample Output 1:

In test case 1, the sum of 'ARR[0]' & 'ARR[2]' is 5 which is greater than 'ARR[1]' which is 2 so the answer is 5.

In test case 2, the sum of 'ARR[0]' and 'ARR[2]' is 6, the sum of 'ARR[1]' and 'ARR[3]' is 10, and the sum of 'ARR[0]' and 'ARR[3]' is 11. So if we take the sum of 'ARR[0]' and 'ARR[3]', it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.

Sample Input 2:

2

5

1 2 3 5 4

9

1 2 3 1 3 5 8 1 9

Sample Output 2:

8

24

Explanation to Sample Output 2:

In test case 1, out of all the possibilities, if we take the sum of 'ARR[0]', 'ARR[2]' and 'ARR[4]', i.e. 8, it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.

In test case 2, out of all the possibilities, if we take the sum of 'ARR[0]', 'ARR[2]', 'ARR[4]', 'ARR[6]' and 'ARR[8]', i.e. 24 so, it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.